

Cinderblock Contract Billing System

Progress Invoice (including Contract Invoice) | Change Order | Payment Schedule | Retainage

Purpose

A consolidated, implementation-oriented source of truth for product, engineering, QA, design, operations, and AI-assisted analysis. It states business invariants, workflow requirements, feature crossover, known gaps, readiness, and a refactor-resistant delivery roadmap.

Document control	Value
Version	1.0
Date	September 15, 2026
Status	Development requirements baseline; open decisions are explicitly flagged
Primary platform	Desktop web workflows
Scope	Business logic and web UI workflow patterns across the four prototype areas
Prototype implementation	Local React state and deterministic fixtures; no production persistence or accounting integration

Normative language

SHALL identifies required behavior. SHOULD identifies a recommended implementation or usability rule. MAY identifies an allowed option. MUST NOT identifies prohibited behavior. GAP identifies a decision that must be resolved before production behavior can be considered complete.

Source basis

This synthesis is grounded in the current repository requirements, prototype state documentation, UI system guidance, implementation models, scenario fixtures, and available model tests. Where the prototypes do not establish production policy, this document records a gap instead of inventing behavior.

Prepared for structured human review and machine-assisted interrogation

Contents

1. Executive requirements baseline	5
1.1 Scope and outcomes	5
1.2 System-wide product invariants	5
1.3 Terminology and canonical definitions	6
2. Cross-feature model and crossover	7
2.1 Source-to-cash relationship	7
2.2 Feature crossover matrix	7
2.3 Billing mode decision table	8
2.4 Event consequences	8
3. Progress Invoice requirements (including Contract Invoice)	9
3.1 Entry, eligibility, and invoice-type selection	9
3.2 Scope selection	9
3.3 Line accounting and calculations	10
3.4 Estimate discount pool	10
3.5 Tracked invoice editor	10
3.6 Contract-aware tracked billing	10
3.7 Customer-facing presentation	10
3.8 Lifecycle and acceptance criteria	11
4. Change Order requirements	12
4.1 Product model	12
4.2 Creation and eligibility	12
4.3 Lifecycle	12
4.4 Acceptance collision with tracked Draft	12
4.5 Contract calculations and representation	13
4.6 Deletion and chained adjustments	13
4.7 Acceptance criteria	13
5. Payment Schedule requirements	14
5.1 Estimate presentation	14
5.2 Manage milestones	14
5.3 On acceptance	14
5.4 Milestone invoice creation	14
5.5 Milestone fulfillment and amounts	15
5.6 Standard Invoice coexistence	15

6. Retainage requirements	16
6.1 Scope and isolation	16
6.2 Defaults and invoice override	16
6.3 Calculation model	16
6.4 Retainage summary and history	16
6.5 Release invoice creation and Draft lock	16
6.6 Saved and customer-facing release invoice	17
6.7 QuickBooks Online accounting contract	17
6.8 Acceptance criteria	17
7. Day-one architecture requirements	18
7.1 Required aggregate boundaries	18
7.2 Persisted facts vs derived read models	18
7.3 Money, percentage, tax, and rounding	18
7.4 Concurrency, atomicity, and idempotency	19
7.5 Snapshot, audit, and mutation policy	19
7.6 Service and API boundaries	19
7.7 Permissions and tier gates	19
8. Missing gaps and decisions	20
8.1 Explicitly deferred scope	21
9. Feature readiness assessment	22
9.1 Weighted scores	22
9.2 Score rationale	22
9.3 Current prototype evidence	22
10. Development roadmap	24
10.1 Phase 0 - Decision closure and executable specification	24
10.2 Phase 1 - Domain foundation	24
10.3 Phase 2 - Progress Invoice and contract-aware mode	24
10.4 Phase 3 - Change Orders	24
10.5 Phase 4 - Payment Schedule	24
10.6 Phase 5 - Retainage	24
10.7 Phase 6 - Integrations and operations	24
10.8 Phase 7 - Hardening and release	25
10.9 What must be known from day one	25
11. Verification strategy and traceability	26

11.1 Test layers	26
11.2 Minimum integrated scenario suite	26
11.3 Requirement traceability convention	26
12. AI analysis guide	27
12.1 Answering precedence	27
12.2 Guardrails for AI-assisted work	27
12.3 High-value questions this document supports	27
12.4 Example structured query template	27
Appendix A. Source inventory and review record	28
A.1 Authoritative repository sources reviewed	28
A.2 Review pass 1 - semantic completeness	28
A.3 Review pass 2 - implementation and analysis fitness	28
A.4 Known evidence limits	28

1. Executive requirements baseline

Core product model

An accepted Estimate establishes the initial contract. Accepted Change Orders revise the contract. Progress Invoices bill accepted contract value over time. Payment Schedule is an advisory planning and history layer. Retainage withholds part of tracked invoice amounts and is later released through a normal accounts receivable Invoice without creating new contract billing.

1.1 Scope and outcomes

- Provide one coherent rule set across prototypes without flattening intentional differences between internal editors and customer-facing documents.
- Treat Contract Invoice as the post-Change Order form of Progress Invoice. Production should use one tracked-invoice domain with contract-aware sources and attribution.
- Separate posted billing from Draft reservations, schedule planning, payments, retainage withholding, and retainage release.
- Define deterministic calculations, lifecycle consequences, source attribution, and transaction boundaries early enough to avoid later data migration and workflow rewrites.
- Identify decisions that the prototypes cannot settle and isolate deferred scope from unresolved core policy.

1.2 System-wide product invariants

SYS-001 Contract value SHALL equal the accepted Estimate total plus net deltas from Accepted Change Orders only.

SYS-002 Draft and Pending Change Orders SHALL NOT alter contract value.

SYS-003 Total invoiced SHALL count posted tracked billing at gross contract-consumption value and SHALL exclude Draft invoices, canceled invoices, payments, and retainage release invoices.

SYS-004 A saved tracked Draft SHALL reserve scope and discount capacity but SHALL NOT update posted billing, Previously Billed, line completion, or Contract progress.

SYS-005 Remaining SHALL be derived from accepted contract value less posted gross tracked billing. Draft reservations may reduce currently available creation scope but SHALL be shown separately.

SYS-006 Payment Schedule SHALL NOT reserve contract value, control Contract progress, or function as an allocation ledger.

SYS-007 Retainage SHALL reduce the amount due on a tracked invoice while leaving its gross contract consumption unchanged.

SYS-008 A Retainage release invoice SHALL create accounts receivable and reduce retained funds held; it SHALL NOT create new contract billing.

SYS-009 At most one active linked tracked Draft may exist for a contract at a time.

SYS-010 Once a Standard Invoice converts an otherwise uninvoiced Estimate, later Progress Invoices, Contract Invoices, and Change Orders SHALL be disabled for that Estimate.

SYS-011 Once tracked Progress Invoice history exists, Standard Invoice creation SHALL be disabled.

SYS-012 Accepted documents and saved invoices SHALL preserve historical values needed to explain prior billing even when current contract structure changes.

1.3 Terminology and canonical definitions

Term	Canonical meaning
Estimate	The originating commercial document. Acceptance establishes the initial contract baseline.
Contract	The accepted Estimate plus all Accepted Change Order deltas, with source and historical line context.
Progress Invoice	A tracked Invoice that bills some or all available accepted scope over time.
Contract Invoice	The same tracked invoice concept after one or more Change Orders are Accepted; it bills the current contract and groups lines by source document.
Standard Invoice	An exclusive full-estimate conversion that copies ordinary invoice lines and does not participate in progress tracking.
Posted billing	Gross tracked billing in a non-Draft, non-Canceled invoice state that contributes to contract progress.
Draft reservation	Scope selected by an active Draft invoice; excluded from posted totals but unavailable to competing creation.
Payment Schedule	Ordered milestones for advisory planning and invoice association; not a value reservation.
Total retained	Cumulative retainage actually withheld from created Progress Invoices.
Retainage released	Cumulative amount placed on created Retainage release invoices, independent of payments received.
Retainage held	Total retained minus Retainage released; the amount still available to release.

2. Cross-feature model and crossover

2.1 Source-to-cash relationship

Layer	Owns	Does not own
Estimate	Initial accepted scope, terms, price, tax/discount context, retainage default, optional schedule	Posted billing ledger or payment settlement
Change Order	Contract delta, adjustment provenance, acceptance state	Invoice payment or independent contract identity
Tracked Invoice	Gross contract allocation, Draft reservation, posted billing, customer A/R amount	Contract mutation
Payment Schedule	Milestone order, plan, milestone-to-invoice association, acceptance automation intent	Contract value reservation or authoritative billing totals
Retainage	Withholding calculation, retained/released/held ledger, release invoice association	New contract consumption when released
Payment	Cash applied to a saved Invoice and its balance due	Contract progress or retainage release recognition

2.2 Feature crossover matrix

Trigger	Progress / Contract Invoice	Change Order	Payment Schedule	Retainage
Estimate accepted	Enables tracked billing	Enables CO creation	Runs configured milestone-1 automation once	Makes Estimate default available to future tracked invoices
Tracked Draft saved	Reserves selected scope; no posted progress	Accepted CO collision cancels it	Milestone is fulfilled only according to the saved-invoice rule; Draft fulfillment policy must be consistent	Reserves chosen rate and amount; does not increase posted total retained until recognition policy fires
Tracked Invoice sent	Gross amount becomes posted billing	May lock Accepted CO financials when draw includes its scope	Associated milestone shows invoice reference and actual amount	Withheld amount increases Total retained
CO accepted	Future creation becomes contract-aware; draft collision is canceled	Delta enters contract value	Future percentages project against current contract; fulfilled actuals remain historical	Default and withholding apply to future tracked invoices
Standard Invoice chosen with schedule	Uses ordinary editor	Disables later COs after conversion	Confirmation removes schedule	No retainage behavior
Retainage release invoice created	No new contract billing	No contract delta	No milestone consumption	Released increases; held decreases; release Draft locks competing release creation

2.3 Billing mode decision table

Contract state	Allowed creation	Required route
Accepted Estimate; no invoices; no Accepted CO	Standard Invoice or Progress Invoice	User first chooses invoice type.
Accepted Estimate; Payment Schedule present; no invoices	Standard Invoice or Progress Invoice	Standard requires destructive-consequence confirmation; Progress opens schedule-aware scope flow.
At least one tracked Progress Invoice exists	Progress Invoice	Standard is disabled.
At least one Accepted Change Order exists	Contract-aware Progress Invoice	Route automatically to contract sources; no separate invoice-type choice between Progress and Contract.
Estimate converted by Standard Invoice	No tracked billing or CO creation	Reopen linked Standard Invoice only.
All accepted scope allocated or reserved	No additional tracked invoice	Creation disabled; existing linked documents remain accessible.

2.4 Event consequences

XF-001 Acceptance, invoice save, invoice posting, CO acceptance, schedule reorder, and retainage release creation SHALL be explicit domain events with idempotency protection.

XF-002 Each event SHALL atomically update its authoritative records and derived read-model invalidation markers.

XF-003 Customer previews SHALL render from saved document snapshots so later Estimate or CO edits cannot rewrite issued documents.

XF-004 Current contract views SHALL derive from accepted source revisions plus posted billing allocations; historical invoice views SHALL derive from invoice snapshots.

XF-005 UI labels may vary by context, but underlying invoice, allocation, reservation, retainage, and payment semantics SHALL remain consistent.

3. Progress Invoice requirements (including Contract Invoice)

Conceptual boundary

Contract Invoice is not a new production document class. It is a Progress Invoice whose billing basis is the accepted contract, including Accepted Change Orders. The UI may call out contract sources, but save, lifecycle, A/R, payment, customer presentation, and reservation rules remain shared.

3.1 Entry, eligibility, and invoice-type selection

PI-001 An accepted, uninvoiced Estimate SHALL offer Create invoice and then What type of invoice? with Standard invoice and Progress invoice.

PI-002 The helper for an invoice type SHALL appear only while that type is selected.

PI-003 The first tracked invoice SHALL offer Percent or amount and Select items. The rest of this estimate/contract SHALL appear only when prior tracked billing exists.

PI-004 After an Accepted CO exists, the tracked creation path SHALL use contract wording, contract value, source-aware eligible lines, and Contract Invoice presentation automatically.

PI-005 When a linked Draft exists, the Estimate SHALL not offer creation of a second Draft. The linked reference SHALL reopen the existing Draft.

PI-006 When no unreserved accepted scope remains, tracked creation SHALL be disabled.

3.2 Scope selection

Method	Input	Allocation result	Validation
The rest of estimate / contract	No amount input	All available scope after posted billing and active Draft reservations	Must have positive remaining available scope
Percent or amount	% or \$ toggle; positive editable value	Distribute requested gross amount across eligible lines without exceeding each line	Value > 0 and <= remaining percentage/amount allowance
Select items	One or more eligible lines	Bill each selected line's full available amount	At least one eligible line; locked/fully consumed lines disabled
Select milestone	Eligible milestone plus invoice-by method	Uses the same percent/amount or select-items implementation	Only unfulfilled milestones without an invoice

PI-007 Changing the %/\$ toggle SHALL clear or reinterpret input only according to an explicit UI rule; submitted values SHALL be validated in currency precision on the server.

PI-008 Percentage mode SHALL use the original accepted billing basis for the active tracked-invoice mode and SHALL enforce the current remaining limit.

PI-009 Dollar allocation SHALL be deterministic, cap every line at available scope, and place any rounding remainder without exceeding the requested amount or a line cap.

PI-010 The editor SHALL display all contract lines for context. Lines with zero This Invoice may be visually subdued in customer views.

PI-011 Progress Invoices SHALL NOT allow arbitrary ad-hoc lines outside accepted contract sources.

3.3 Line accounting and calculations

Metric	Required formula / rule
Line contract amount	Accepted source quantity x accepted source unit price plus any source-level financial treatment assigned to that line by production allocation policy.
Previously billed	Sum of posted gross allocations to the stable contract line. Exclude Draft and Canceled allocations.
Draft reserved	Sum of active Draft gross allocations to the line.
Line available	$\max(0, \text{line contract amount} - \text{previously billed} - \text{Draft reserved})$, adjusted for supersession rules.
Line % Complete	$(\text{Previously billed} + \text{This Invoice gross}) / \text{line contract amount} \times 100$. For historical, removal, or zero-value lines, show an intentional non-editable treatment.
Contract progress	Total posted gross tracked billing / accepted contract value.
Invoice amount due before payments	Gross current billing - allocated discount + tax + eligible charges - retainage withheld.

3.4 Estimate discount pool

PI-012 An Estimate-level discount SHALL be represented as a finite pool consumed across tracked invoices, never duplicated in full on every invoice.

PI-013 Creation SHOULD suggest a proportional discount based on selected gross billing divided by remaining gross contract basis.

PI-014 The user MAY edit the suggestion from zero through the remaining pool. Values outside that range SHALL be rejected.

PI-015 Draft discount allocation SHALL reserve pool capacity; posting SHALL consume it; canceling SHALL release it.

PI-016 The production model SHALL preserve the accepted Estimate discount and each invoice's applied portion so totals remain auditable.

3.5 Tracked invoice editor

PI-017 The editor SHALL show Item name, Qty, Contract Amount, Previously Billed, This Invoice, and % Complete.

PI-018 Qty and accepted source context SHALL be read-only. Draft current amounts SHALL be editable only within available line capacity.

PI-019 Apply remaining SHALL fill the line's current available amount through the same calculation and validation path as manual input.

PI-020 Draft save SHALL create or update one local/production Draft and keep the user on that Invoice. Closing unsaved creation SHALL not mutate contract totals.

PI-021 Sent, Partially paid, and Paid tracked invoices SHALL reopen read-only unless future product policy explicitly permits amendment.

PI-022 Estimate documents SHALL keep ordinary estimate columns and SHALL NOT gain line-level billing columns.

3.6 Contract-aware tracked billing

CI-001 The contract basis SHALL include the Accepted Estimate and every Accepted CO, each with a stable source document identifier and revision.

CI-002 Editor and customer tables SHALL group or label lines by original Estimate and Change Order source.

CI-003 Superseded original lines and negative removal adjustments SHALL remain visible as historical contract context but SHALL not accept new billing.

CI-004 Positive replacement or added lines SHALL carry their own billable capacity. Prior billing on a superseded source SHALL remain historically attributed.

CI-005 A reduction that makes posted billing exceed revised contract value SHALL surface an over-invoiced state and SHALL block additional billing until resolved.

CI-006 The production domain SHALL preserve the semantic link between an adjustment and its source line; labels alone are insufficient.

3.7 Customer-facing presentation

PI-023 PDF and web invoice views SHALL use the same saved financial snapshot and show consistent values.

PI-024 Tracked invoice customer tables SHALL show contract context and current billing without exposing internal cost, markup, margin, profit, or prototype controls.

PI-025 Web preview MAY allow Show all contract items, default on; filtering SHALL not alter invoice math.

PI-026 Payment, print, and delivery actions in production SHALL operate on saved invoice identity and SHALL be permission and status aware.

3.8 Lifecycle and acceptance criteria

State	Financial edit	Counts as posted billing	Reserves scope	Typical next action
New / unsaved	Yes	No	No durable reservation	Save Draft or close
Draft	Yes	No	Yes	Send, cancel, or continue editing
Sent	No	Yes	Consumed as posted	Record payment
Partially paid	No	Yes	Consumed as posted	Record remaining payment
Paid	No	Yes	Consumed as posted	View history
Canceled	No	No	No	View audit history

- Verify each creation method in unbilled, partially billed, fully billed, individually completed, discount, and discount-partially-used scenarios.
- Verify prior posted values remain immutable while current Draft edits and that Draft reservations cannot be double-selected.
- Verify accepted CO routing, source grouping, superseded rows, line caps, and over-invoiced blocking.
- Verify saved Draft reopening, one-Draft enforcement, posted locks, customer PDF/web parity, and close-without-save behavior.

4. Change Order requirements

4.1 Product model

CO-001 A Change Order SHALL be an Estimate-style contract adjustment linked to one originating Estimate/contract.

CO-002 A CO SHALL contain additive lines and explicit negative-quantity adjustment lines. Negative lines SHALL retain source provenance.

CO-003 Creating an adjustment from an Estimate line SHALL add a locked negative copy of the selected source plus a separate editable positive line for replacement or new scope.

CO-004 The prototype has no implicit replacement relationship. Production SHALL either preserve this independent-line behavior or explicitly add a typed replacement relation before implementation.

CO-005 PO Number and Customer Reference SHALL be optional, document-specific, and not inherited automatically.

4.2 Creation and eligibility

CO-006 CO creation SHALL require an Accepted Estimate that was not exclusively converted by Standard Invoice.

CO-007 Eligible source lines SHALL exclude scope already reserved by another open CO adjustment or unavailable under accepted downstream adjustments.

CO-008 A new CO SHALL begin as unsaved working state and become Draft only after Save.

CO-009 Save SHALL require at least one meaningful non-zero adjustment and all mandatory line data.

CO-010 CO numbering SHALL be sequential within the contract and SHALL not be reused after deletion or cancellation unless a formal numbering policy says otherwise.

4.3 Lifecycle

State	Contract effect	Reservation	Editable	Allowed consequences
Unsaved	None	None durable	Yes	Save or close
Draft	None	Selected source scope	Yes	Submit/send, delete
Pending	None	Selected source scope	Yes under prototype rules	Accept, decline, cancel, delete
Accepted	Net delta enters contract	Becomes accepted scope	Only until locked by posted draw; edits return to Pending	Create contract-aware tracked invoice
Declined	None	Released	No	Historical view
Canceled	None	Released	No	Historical view

CO-011 Accepting a CO SHALL record accepted timestamp, actor, accepted revision, and immutable financial snapshot.

CO-012 Editing accepted financial terms before a posted draw SHALL create a new pending revision while preserving the prior accepted revision for audit.

CO-013 Once posted billing includes CO scope, its accepted financial fields SHALL lock.

CO-014 Non-Accepted COs MAY be deleted. Accepted COs SHALL NOT be deleted.

4.4 Acceptance collision with tracked Draft

CO-015 If a tracked Draft exists when a CO is Accepted, the acceptance transaction SHALL cancel that Draft, release scope and discount reservations, retain it in read-only history, and require a new invoice against the updated contract.

CO-016 The user SHALL receive a clear acceptance-result notification identifying the canceled Draft.

CO-017 The acceptance and Draft cancellation SHALL be atomic so the contract cannot enter a state with stale reservations.

4.5 Contract calculations and representation

Metric	Rule
CO line amount	Unit price x signed quantity.
Cost Plus	Applied only to eligible positive cost according to current prototype behavior; negative source adjustments do not create positive Cost Plus.
Tax	Applied to the documented taxable base after Cost Plus and discount. Exact jurisdiction and rounding policy is a production gap.
CO impact	Signed line subtotal + eligible Cost Plus - discount + tax.
Contract value	Accepted Estimate total + sum(Accepted CO impacts).
Open CO adjustment	Draft/Pending impact shown separately; excluded from accepted contract value.

4.6 Deletion and chained adjustments

CO-018 Deleting a non-Accepted CO SHALL release its source reservations and remove its working relationships.

CO-019 If later non-Accepted COs depend on a deleted CO's positive lines, deletion SHALL either cascade with explicit confirmation or be blocked. The production policy is a GAP.

CO-020 Accepted adjustment history SHALL remain stable even when later COs adjust earlier CO lines.

4.7 Acceptance criteria

- Verify addition-only, original-line adjustment, CO-line adjustment, positive and deductive totals, optional references, and save validation.
- Verify every lifecycle transition, source reservation, deletion restriction, accepted revision history, and financial lock after posted billing.
- Verify Contract details order, linked invoice history, contract summary, accepted-only contract value, and explicit Draft reservations.
- Verify over-invoiced state after deductive acceptance and that no further tracked billing can increase the imbalance.

5. Payment Schedule requirements

Product hierarchy

Contract progress answers how much has actually been billed. Payment Schedule answers which billing milestones have happened and what is planned next. The schedule is advisory and must not compete with the authoritative Contract progress block.

5.1 Estimate presentation

PS-001 Payment Schedule SHALL appear as an Estimate-level prototype area and SHALL not create a left-navigation document type.

PS-002 The schedule table SHALL show ordered milestone name, percentage, projected or actual amount, date, and Invoice reference/action.

PS-003 Fulfilled milestones SHALL show a restrained green check-circle before the name, a subtle completed background, and a clickable Invoice # reference.

PS-004 Unfulfilled milestones SHALL retain normal styling and Create invoice.

PS-005 The header action SHALL read Manage milestones and open the existing schedule editor.

PS-006 The schedule SHALL NOT show allocated/scheduled tallies, progress bars, 100% warnings, or contract billing totals.

5.2 Manage milestones

PS-007 Every manually created milestone SHALL have a nonblank name before Save changes is enabled.

PS-008 Percentage and date MAY be optional. The UI SHALL not auto-generate Invoice N names in the schedule editor.

PS-009 Rows SHALL be reorderable by a functional drag handle. Up/Down buttons SHALL not coexist with drag-and-drop.

PS-010 A drop SHALL preserve all values, immediately renumber rows, and update every consumer of sequence order.

PS-011 Delete SHALL remain at the right edge. Removal of an invoiced milestone requires a production policy and is a GAP; prototype behavior must not erase invoice history.

5.3 On acceptance

PS-012 On acceptance SHALL appear only while the Estimate is Draft or Pending and SHALL be hidden at Accepted or later states.

PS-013 Options SHALL be Do nothing, Create draft invoice for milestone 1, and Create and send invoice for milestone 1.

PS-014 Milestone 1 SHALL be resolved from current order at acceptance time; the system SHALL not retain a hidden reference to a previously first milestone.

PS-015 Automatic creation SHALL use the full amount of milestone 1 and the existing tracked Progress Invoice and retainage rules.

PS-016 Acceptance automation SHALL be idempotent. Retried acceptance SHALL not create duplicate invoices.

PS-017 An automatic action SHALL not be saveable when milestone 1 has no valid positive amount. If an active tracked Draft conflicts at acceptance, acceptance SHALL remain successful, the existing Draft SHALL remain unchanged, the milestone SHALL remain unfulfilled, and an inline error SHALL explain the conflict.

5.4 Milestone invoice creation

PS-018 Progress Invoice scope SHALL continue to offer Select milestone, The rest of this estimate/contract when eligible, Percent or amount, and Select items.

PS-019 Select milestone SHALL use one compact combined screen with a Milestone dropdown and Invoice by choices. No separate invoice-method step follows.

PS-020 The dropdown SHALL include current sequence numbers, such as 2. Rough-in, and only eligible unfulfilled milestones without an invoice.

PS-021 Percent or amount SHALL be selected by default whenever the screen opens or a milestone changes.

PS-022 A scheduled percentage SHALL default to % and populate the editable value. A milestone without a percentage SHALL default to %, show a blank input, and never inherit a previous milestone's value.

PS-023 Select items SHALL invoke the exact shared tracked-invoice item-selection flow and validation.

PS-024 Direct row Create invoice SHALL skip the initial scope question, open the same Select milestone screen preselected to that row, and allow changing the milestone.

5.5 Milestone fulfillment and amounts

State	Displayed amount	Contract effect
Unfulfilled percentage milestone	Current accepted contract value x scheduled percentage	Advisory only; does not reserve scope
Unfulfilled no-percentage milestone	Blank or intentionally unspecified	Advisory only
Fulfilled milestone	Actual saved invoice gross association amount	Invoice itself controls billing; schedule retains history
Accepted CO after prior fulfillment	Past fulfilled actual remains fixed; future percentage projections use current contract	Accepted CO alters contract, not past invoice history

PS-025 Saving a milestone-associated invoice SHALL store milestone ID and milestone-name snapshot on the invoice association.

PS-026 Ad-hoc Progress Invoice milestone naming MAY be blank; on save the system SHALL generate Invoice 1, Invoice 2, etc. This fallback SHALL not apply in Manage milestones.

PS-027 Scheduled percentage is advisory and editable during creation. Actual invoice constraints remain the authoritative contract limits.

5.6 Standard Invoice coexistence

PS-028 A schedule SHALL not lock the Estimate into tracked invoicing. What type of invoice? SHALL preserve Standard invoice and Progress invoice.

PS-029 Selecting Standard invoice while a schedule exists SHALL show Create standard invoice? with the stated schedule-removal consequence.

PS-030 Cancel SHALL return to invoice-type selection without mutation.

PS-031 Create invoice SHALL remove the schedule and open the existing Standard Invoice editor with the full Estimate copied.

PS-032 Saving SHALL use existing Standard conversion behavior, preserve Linked to Invoice #..., and keep the user on the saved invoice.

PS-033 GAP: decide whether schedule removal occurs immediately at editor entry or commits with invoice save. Production SHOULD make the removal and conversion one atomic save transaction to avoid data loss if creation is abandoned.

6. Retainage requirements

6.1 Scope and isolation

RT-001 Retainage SHALL be an explicit Estimate prototype area that layers on existing tracked billing; ordinary Progress Invoice scenarios SHALL not display Retainage UI.

RT-002 The default entry SHALL be Retainage - Not yet invoiced: retainage configured, no Progress Invoices, and otherwise unchanged Estimate UI.

RT-003 Retainage SHALL apply only to tracked Progress/Contract Invoices, never Standard Invoice conversion.

6.2 Defaults and invoice override

RT-004 The Estimate SHALL store a future-invoice retainage default rate.

RT-005 Each Draft tracked invoice SHALL snapshot its own retainage rate and MAY override the default without mutating the Estimate.

RT-006 Changing the Estimate default SHALL affect future invoices only and SHALL not recalculate saved invoices.

RT-007 The retainage rate SHALL be validated within the allowed product range. The maximum and permission model are production GAPs.

6.3 Calculation model

Metric	Formula / recognition
Gross current billing	Sum of current tracked invoice allocations before retainage.
Retainage withheld on invoice	Round(gross eligible billing x invoice retainage rate) under the defined currency rounding policy.
Tracked invoice amount due	Normal invoice subtotal/tax treatment less retainage withheld, subject to the final tax-base policy.
Total retained	Cumulative retainage actually withheld from recognized Progress/Contract Invoices. Never theoretical rate x full contract.
Retainage released	Cumulative face amount of created Retainage release invoices; not payments received.
Retainage held	$\max(0, \text{Total retained} - \text{Retainage released})$.
Contract progress	Gross posted tracked billing / accepted contract value; unaffected by withholding or release.

Example

A \$75,000 cumulative gross draw at 10% creates \$7,500 Total retained. A \$2,500 release leaves Total retained at \$7,500, increases Retainage released to \$2,500, and reduces Retainage held to \$5,000.

6.4 Retainage summary and history

RT-008 The summary labels SHALL be Total retained, Retainage released, and Retainage held.

RT-009 Total retained SHALL not reset or decrease when release invoices are created.

RT-010 Contract history SHALL distinguish tracked draw invoices from Retainage release invoices while retaining normal Invoice identity and links.

6.5 Release invoice creation and Draft lock

RT-011 Release creation SHALL be available only when Retainage held is greater than zero and no active Retainage release Draft exists.

RT-012 The creation/editor section SHALL show Retainage held and editable Amount to release. It SHALL omit a redundant Retainage released / Total block.

RT-013 Amount to release SHALL default from entry selection, allow partial release, be greater than zero, and not exceed Retainage held.

RT-014 Saving a Retainage release SHALL create a normal A/R Invoice linked to the Estimate/contract and keep the user on the saved Draft.

RT-015 A saved release Draft SHALL reserve its release amount and block another release Draft. Canceling SHALL release that reservation.

RT-016 Retainage released SHALL increase when Save invoice creates the Retainage release Draft, not according to send or customer payment status.

RT-017 A release invoice SHALL not consume contract line capacity, change Total invoiced, change line completion, fulfill a Payment Schedule milestone, or modify accepted contract value.

6.6 Saved and customer-facing release invoice

RT-018 Customer PDF and web invoice content SHALL show Retainage release, secondary linked Estimate # dynamically, and the release amount once as invoice content.

RT-019 The totals area SHALL show normal Total and Balance Due, plus Amount Paid only when actual payments exist. It SHALL not repeat a Retainage released row.

RT-020 Previously withheld retainage SHALL never be represented as Amount Paid.

RT-021 Customer views SHALL omit retainage rate, Total retained, held/released-to-date metrics, contract lines, Payment Schedule milestones, and internal accounting detail.

RT-022 The internal saved invoice MAY retain useful retainage context while normal A/R totals and payment behavior remain authoritative.

6.7 QuickBooks Online accounting contract

RT-QBO-001 QBO integration SHALL preserve gross revenue/billing, retainage receivable or withholding treatment, release transfer, customer A/R, and payment application without double-counting revenue.

RT-QBO-002 The integration SHALL use stable mappings for service items, retainage asset accounts, tax codes, customers/jobs, Estimate/Invoice references, and external IDs.

RT-QBO-003 Create/update operations SHALL be idempotent and reconcile partial failures. The local ledger SHALL remain authoritative for workflow state.

RT-QBO-004 The current prototype documents QBO requirements only; no QBO behavior is implemented. Account selection, tax behavior, sync direction, edit policy, and error recovery remain production gaps.

6.8 Acceptance criteria

- Verify default and per-invoice override, gross scope consumption, rounding, discount/tax interaction, and Total retained accumulation across multiple tracked invoices.
- Verify partial and full releases, release Draft reservation, canceled Draft unlock, no negative held balance, and no contract-progress mutation.
- Verify customer PDF/web single content line, dynamic Estimate reference, normal A/R totals, and payment-only Amount Paid.
- Verify Payment Schedule milestones use gross tracked amounts and that release invoices never fulfill milestones.

7. Day-one architecture requirements

Refactor avoidance rule

Persist source facts and event outcomes; derive summaries. Use stable identities and versioned snapshots. Do not encode contract progress, schedule progress, retainage held, or invoice availability as independently editable totals.

7.1 Required aggregate boundaries

Aggregate / record	Required responsibilities	Key identities
Contract	Origin Estimate, accepted revision set, accepted contract value read model, status	contract_id, estimate_id
Contract source revision	Immutable accepted Estimate or CO financial snapshot	source_document_id, revision_id, accepted_at
Contract line	Stable lineage across source documents, signed value, billable/superseded state	contract_line_id, source_line_id, adjusts_line_id
Invoice	Document identity, kind, lifecycle, customer/A/R totals, immutable issued snapshot	invoice_id, contract_id, invoice_kind
Invoice allocation	Gross tracked amount assigned to one contract line	invoice_id, contract_line_id
Reservation	Draft-held contract line, discount pool, or retainage-release capacity	reservation_id, owner_invoice_id, resource_key
Payment schedule	Versioned ordered milestones and acceptance action	schedule_id, contract_id, version
Milestone	Stable identity, sequence, name, optional percentage/date, fulfillment association	milestone_id, schedule_id
Retainage entry	Withhold, release, cancel/reversal events and source invoice	retainage_entry_id, invoice_id, event_type
Payment	Actual cash application to Invoice	payment_id, invoice_id, external_id

7.2 Persisted facts vs derived read models

Persist as fact	Derive for display / validation
Accepted document revisions and signed lines	Current accepted contract value
Invoice lifecycle events and gross line allocations	Previously billed, Total invoiced, line completion, Remaining
Active/canceled reservations	Available scope and reserved-by-Draft messaging
Retainage rate snapshot and withhold/release entries	Total retained, released, and held
Milestone stable ID, sequence, and invoice association	Numbered dropdown labels, projected amount, completed-state styling
Payment applications	Amount Paid and Balance Due

7.3 Money, percentage, tax, and rounding

ARC-001 Money SHALL use fixed-precision decimal or integer minor units; binary floating point SHALL not be authoritative.

ARC-002 Every percentage SHALL store both a precise rate representation and its calculation basis.

ARC-003 Allocation SHALL define rounding precision, deterministic remainder placement, and reconciliation to document totals.

ARC-004 Tax SHALL record jurisdiction, tax code, taxable basis, rounding scope, and source snapshots. Retainage tax treatment is a launch-blocking decision.

ARC-005 Discount allocation SHALL reconcile across invoices to the accepted discount pool without overconsumption.

7.4 Concurrency, atomicity, and idempotency

ARC-006 Draft creation SHALL acquire reservations using optimistic concurrency or transactional locking against the current contract version.

ARC-007 CO acceptance plus conflicting Draft cancellation SHALL commit atomically.

ARC-008 Estimate acceptance plus milestone automation SHALL use one idempotency key and record completion/failure for safe retries.

ARC-009 Invoice posting SHALL atomically transition reservations to posted allocations and create retainage entries.

ARC-010 Release invoice creation SHALL atomically validate held capacity and create its release reservation/entry.

ARC-011 External delivery and QBO synchronization SHALL use an outbox/event delivery pattern so local state and side effects can recover independently.

7.5 Snapshot, audit, and mutation policy

ARC-012 Issued invoice rendering SHALL use an immutable invoice snapshot, including customer, job, source labels, line descriptions, dates, taxes, terms, and totals.

ARC-013 Every accepted Estimate/CO revision SHALL be immutable and attributable to actor and timestamp.

ARC-014 Every cancellation, void, release, and external sync correction SHALL be an auditable event; hard deletion SHALL be limited to allowed unsent working records.

ARC-015 Derived summary read models SHALL be rebuildable from facts and SHALL carry a source version for stale-read detection.

7.6 Service and API boundaries

- Contract service: accepted sources, line lineage, contract value, over-invoiced state.
- Billing service: eligibility, allocation, reservations, invoice lifecycle, discount pool, customer snapshot.
- Schedule service: schedule version, milestone order, acceptance automation intent, milestone association.
- Retainage service/ledger: default, invoice snapshot rate, withholding, release capacity, release history.
- Payment/A/R service: payments, balance, refunds/credits when defined; independent of contract progress.
- Integration adapters: invoice delivery, document rendering, and QBO, each idempotent and observable.

7.7 Permissions and tier gates

ARC-016 Authorization SHALL be enforced server-side for accepting Estimates/COs, editing accepted financials, creating/sending invoices, releasing retainage, recording payments, and changing integration mappings.

ARC-017 Feature tiering SHALL be policy-driven. The repository contains a Retainage prototype while earlier Change Order notes described retainage as deferred/High Rise; product must resolve entitlement before schema or UI gating is finalized.

ARC-018 Audit records SHALL include user, organization, effective contract version, request idempotency key, before/after state, and reason where required.

8. Missing gaps and decisions

The following are not safe to infer from frontend prototypes. Priority P0 blocks durable domain design or accounting correctness; P1 blocks production release or reliable operations; P2 can be deferred behind a stable boundary.

ID	Pri	Area	Unresolved question	Required closure
GAP-001	P0	Invoice lifecycle	Exact Draft -> Sent -> Partially paid -> Paid -> void/cancel transitions; amendment and return-to-Draft policy.	Define state machine, permissions, reversal effects, and audit events.
GAP-002	P0	Recognition point	Whether posted billing and Total retained recognize on send, finalization, or another accounting event.	Choose one canonical posting event and map all summaries and integrations to it.
GAP-003	P0	Tax and retainage	Whether retainage is calculated before/after tax and how tax is recognized on withholding/release by jurisdiction.	Accounting and tax review with worked examples and QBO mapping.
GAP-004	P0	CO revision	How edits to an Accepted CO are approved and how old/new accepted revisions coexist.	Define revision state machine and snapshot semantics.
GAP-005	P0	Deductive CO recovery	What action resolves a contract that becomes over-invoiced after a reduction.	Define credit memo, refund, rebalance, or manual-resolution workflow.
GAP-006	P0	Line allocation	How document-level discounts, Cost Plus, and tax are attributed to contract lines for % Complete and source totals.	Define allocation basis and rounding/remainder rules.
GAP-007	P0	Schedule removal	Schedule is removed on Standard Invoice editor entry in prototype language; abandon/failed-save semantics are unclear.	Commit schedule removal atomically with successful Standard Invoice conversion.
GAP-008	P1	Acceptance delivery	For Create and send invoice for milestone 1, external send may fail after Estimate acceptance and invoice creation.	Use an outbox and define visible delivery status, retry authority, and notification without duplicating the invoice.
GAP-009	P1	Payments	Payment application, overpayment, refund, write-off, failed payment, and reversal behavior.	Define A/R ledger and invoice balance policy separately from contract billing.
GAP-010	P1	CO dependency deletion	Deletion behavior when another Draft/Pending CO adjusts the first CO.	Block or explicit cascade; never orphan lineage.
GAP-011	P1	Milestone history	Rename/reorder/delete behavior after invoices are associated.	Preserve stable ID plus invoice-side name/sequence snapshot; define editor restrictions.
GAP-012	P1	Retainage rates	Allowed max, precision, per-user permissions, and edits after Draft save.	Approve validation and mutation rules.
GAP-013	P1	QBO sync	Accounts, items, tax codes, customer/job mapping, edit ownership, retries, duplicate prevention, and reconciliation.	Approve mapping contract and failure runbook before implementation.
GAP-014	P1	Multi-user concurrency	Simultaneous Drafts, CO acceptance, schedule edits, and release attempts.	Version all aggregates; design conflict messages and retry behavior.
GAP-015	P1	Permissions and tiers	Who may accept, send, release, edit accepted documents, and access Retainage.	Define RBAC/entitlement matrix.
GAP-016	P1	Document numbering	Reservation, gaps, retries, environment collisions, and cancellation policy for Estimate/CO/Invoice numbers.	Define organization-scoped sequence and audit rules.
GAP-017	P2	Currency/localization	Prototype assumes USD, en-US, and simple dates.	Keep currency/time-zone fields in schema even if launch is single-currency.
GAP-018	P2	Accessibility	Keyboard drag-and-drop, focus management, announcements, and document accessibility are unverified.	Set WCAG target and acceptance tests.

8.1 Explicitly deferred scope

Deferred area	Boundary / reason	Design accommodation now
Mobile and responsive optimization	Current emphasis is desktop web business logic and workflow completeness.	Use semantic responsive components and avoid desktop-only data contracts.
Native mobile apps	Outside prototype scope.	Expose capability-oriented APIs without browser assumptions.
Offline operation	Requires conflict and sync product design.	Use versioned APIs and idempotent mutations.
Advanced reporting and analytics	No production reporting requirements established.	Retain event timestamps, actors, source IDs, and rebuildable ledger facts.
Multi-currency and exchange rates	Prototype is USD only.	Store currency on contract and every monetary document; reject mixed-currency contracts initially.
Localization and international tax	No locale/jurisdiction set defined.	Avoid formatted strings as stored values; retain tax-code abstraction.
Stored materials and specialized construction billing	Not represented in prototypes.	Keep invoice allocation extensible through typed components, not extra ad-hoc columns.
Production e-signature and customer acceptance	Prototype acceptance controls are local.	Model acceptance as an idempotent event with actor and evidence fields.
Production payment processing	Preview payment controls are inert.	Keep payment ledger separate from invoice and contract ledgers.
Production PDF generation and delivery	Prototype previews are React-rendered; actual generation/delivery is absent.	Use the same saved invoice snapshot for HTML, PDF, and email delivery.
QBO implementation	Requirements exist; connector behavior is not built.	Finalize mappings and outbox boundaries before implementation.
Bulk operations, import/export, and migration UI	No product behavior defined.	Design durable IDs and source snapshots to support later migration.

9. Feature readiness assessment

Scores evaluate requirement clarity and production preparedness, not engineering effort or commercial priority. Each dimension is scored 0-5 and weighted: business rules 25%, workflow coverage 20%, cross-feature consistency 15%, data/architecture definition 15%, test evidence 15%, integration/operations 10%.

Band	Meaning
85-100	Implementation-ready with only bounded operational decisions
70-84	Strong prototype baseline; close named gaps before production build
50-69	Useful validated concept; significant domain or operational policy remains
Below 50	Exploratory; not ready for production planning

9.1 Weighted scores

Feature	Rules	Flow	Cross	Data	Tests	Ops	Score
Progress Invoice + Contract mode	4.5	4.5	4.2	3.4	3.8	1.8	78
Change Order	4.3	4.2	4.2	3.3	1.5	1.6	69
Payment Schedule	4.4	4.4	4.3	3.0	1.2	1.5	68
Retainage	4.4	4.3	4.3	3.1	2.2	1.2	70
Integrated platform	4.4	4.3	4.3	3.2	2.2	1.5	71

9.2 Score rationale

Feature	What is ready	What holds the score back
Progress + Contract	Broad scenario set, deterministic allocation model, model tests, Draft reservation rules, customer previews, accepted-CO route.	Posting policy, production allocation/tax/discount precision, permissions, persistence, integration, and full lifecycle are open.
Change Order	Rich lifecycle and adjustment requirements, contract crossover, collision handling, over-invoiced scenario.	No dedicated model tests found; accepted revision mechanics, cascade policy, deductive recovery, and production permissions remain open.
Payment Schedule	Detailed milestone editor, sequence, acceptance automation, invoice crossover, Standard coexistence, and hierarchy rules.	No dedicated model tests found; automation recovery, post-association edit/delete policy, transactional schedule removal, and persistence are open.
Retainage	Clear withholding/release math, editor/customer distinctions, release Draft lock, cross-feature rules, and some Progress model tests.	Accounting recognition, tax treatment, QBO mapping, permissions, reversal behavior, and standalone retainage service tests are open.

Readiness conclusion

The combined requirements are strong enough to begin architecture and decision-closure work. They are not yet safe for direct production implementation without resolving the P0 gaps, especially posting recognition, tax/retainage basis, accepted revision semantics, deterministic financial allocation, and transactional automation behavior.

9.3 Current prototype evidence

- Progress Invoice model tests cover creation methods, seeded invoice consistency, prior-billing immutability, line limits, percentage basis, discount pool, and retainage behavior.
- Repository scripts provide TypeScript checking, ESLint, Vite production build, and a Node model-test command for Progress Invoices.

- Change Order, Contract Invoice, and Payment Schedule modules contain deterministic local models and interactive scenarios, but no dedicated automated test files were found in the current tree.
- All prototypes use local React state. No API, persistence, production PDF generation, delivery, payment processor, accounting system, or QBO connector is implemented.

10. Development roadmap

The roadmap is sequenced by dependency. Parallel UI work is safe only after the shared domain contracts for that phase are frozen. Exit criteria are more important than dates.

10.1 Phase 0 - Decision closure and executable specification

- Resolve every P0 gap with approved examples, formulas, lifecycle diagrams, and reversal behavior.
- Approve canonical tracked-invoice terminology: one Progress Invoice domain with estimate and contract modes.
- Define posting/recognition, tax and retainage bases, discount/Cost Plus allocation, money rounding, and over-invoiced recovery.
- Create a requirement-to-test traceability matrix using the IDs in this document.
- Exit: no P0 business decision is delegated to frontend or integration implementation.

10.2 Phase 1 - Domain foundation

- Implement Contract, accepted source revisions, stable contract-line lineage, Invoice, allocation, reservation, payment, schedule, milestone, and retainage ledger schemas.
- Implement decimal money, deterministic allocation/rounding, optimistic concurrency, idempotency keys, audit events, and rebuildable read models.
- Publish versioned service/API contracts and event schemas. Add contract tests and migration strategy.
- Exit: core calculations pass golden examples and concurrency tests without any UI dependency.

10.3 Phase 2 - Progress Invoice and contract-aware mode

- Build eligibility, scope selection, line allocation, one-Draft reservations, discount pool, invoice lifecycle, and Standard conversion exclusivity.
- Add source-aware contract mode and historical/superseded line presentation behind the same Invoice aggregate.
- Build internal editor, saved invoice snapshot, customer web view, and PDF renderer from shared presentation data.
- Exit: all PI/CI requirements and scenario tests pass, including multi-user reservation races and close-without-save behavior.

10.4 Phase 3 - Change Orders

- Build CO creation, signed adjustments, source lineage, lifecycle, accepted revision snapshots, reservations, and deletion rules.
- Integrate acceptance with atomic Draft cancellation and automatic contract-aware invoice routing.
- Implement over-invoiced detection and the approved recovery boundary.
- Exit: accepted contract reconstruction and historical invoices reconcile across chained CO scenarios.

10.5 Phase 4 - Payment Schedule

- Build schedule versioning, required names, keyboard-accessible drag reorder, sequence projections, milestone invoice association, and completed history styling.
- Integrate shared tracked-invoice scope components; do not fork Percent/Amount or Select Items logic.
- Implement idempotent acceptance automation, conflict recovery, and atomic Standard conversion/schedule removal.
- Exit: ordering, fulfilled actuals, accepted-CO projections, automation retries, and all crossover scenarios reconcile.

10.6 Phase 5 - Retainage

- Implement Estimate default, invoice snapshot override, withholding ledger, release capacity, release Draft reservation, and summary read model.
- Build release editor and saved/customer invoice views while preserving normal A/R payment semantics.
- Test Progress, Contract, Payment Schedule, discounts, tax, COs, cancellations, and releases together.
- Exit: retained/released/held ledger reconciles from events and never changes gross Contract progress.

10.7 Phase 6 - Integrations and operations

- Implement invoice delivery, web/PDF artifact storage, payment events, QBO adapter, outbox, idempotent retries, reconciliation UI, and operational alerts.
- Add RBAC, tier entitlements, security review, audit exports, support diagnostics, and data retention policy.
- Exit: partial failures are recoverable without manual database edits and financial reconciliation is explainable end to end.

10.8 Phase 7 - Hardening and release

- Run migration rehearsal, load/concurrency testing, accessibility validation, browser coverage, financial golden tests, and end-to-end acceptance suites.
- Pilot with seeded and anonymized real-world contracts, including deductive COs, multiple draws, schedule reorder, partial retainage release, payment, and QBO retry.
- Exit: readiness score is reassessed at 85+ with no unresolved P0/P1 correctness gap and signed operational runbooks.

10.9 What must be known from day one

Decision	Why late discovery causes refactoring
One tracked-invoice aggregate with contract mode	Separate Progress/Contract implementations duplicate lifecycle, reservation, payment, snapshot, and customer logic.
Posting recognition event	Every total, retainage entry, milestone fulfillment, lock, and integration depends on it.
Stable line lineage and accepted revisions	Labels and mutable rows cannot reconstruct historical contracts or invoice attribution.
Gross billing vs net A/R separation	Retainage and payments otherwise corrupt Contract progress and Remaining.
Reservation as first-class data	One-Draft rules and concurrency cannot be reliably enforced from UI state.
Deterministic financial allocation	Discount, tax, Cost Plus, and rounding must reconcile at line, invoice, contract, and QBO levels.
Schedule as advisory	Treating percentages as allocation creates false constraints and conflicts with ad-hoc draws.
Invoice snapshots	Mutable Estimate/CO/customer data must not rewrite issued documents.
Idempotent event/outbox boundaries	Acceptance automation, send, payment, and QBO retries otherwise create duplicates.
Cancellation/reversal policy	Deleting or editing financial facts breaks audit and integration reconciliation.

11. Verification strategy and traceability

11.1 Test layers

Layer	Required evidence
Pure calculation tests	Golden money examples for allocation, rounding, discount pool, tax, retainage, release, contract value, and over-invoiced states.
State-machine tests	Allowed/forbidden transitions, permissions, accepted revisions, locks, cancellations, reversals, and retries.
Repository/service tests	Atomic reservation, concurrent edits, idempotency, snapshot persistence, read-model rebuild, and migration.
Component tests	Validation, default selection, milestone switching, drag reorder, conditional sections, locked controls, and error recovery.
End-to-end tests	Estimate acceptance through billing, CO crossover, schedule automation, retainage withholding/release, payment, PDF/web view, and QBO retry.
Visual/accessibility tests	Reference fidelity, keyboard paths, focus order, screen-reader names, contrast, print/PDF layout, and responsive safety.

11.2 Minimum integrated scenario suite

E2E-001 Accepted Estimate -> first partial Progress Draft -> send -> second rest-of-estimate invoice -> Converted.

E2E-002 Accepted Estimate with discount -> multiple draws -> exact discount pool exhaustion without rounding drift.

E2E-003 Tracked Draft -> CO Accepted -> Draft canceled and reservations released -> new contract-aware Draft.

E2E-004 Accepted CO adjusts original and CO-added lines -> grouped invoice retains historical/superseded context.

E2E-005 Deductive CO creates over-invoiced contract -> additional billing blocked -> approved recovery restores consistency.

E2E-006 Draft Estimate with schedule -> reorder -> acceptance creates invoice from current milestone 1 exactly once.

E2E-007 Milestone switch from percentage to no-percentage clears inherited value and keeps Percent or amount selected.

E2E-008 Schedule present -> Standard Invoice confirmation cancel preserves schedule; confirmed saved conversion removes schedule atomically.

E2E-009 Three tracked invoices withhold retainage -> partial release -> later draw -> full release; gross progress and all three retainage metrics reconcile.

E2E-010 Release Draft blocks a second release; cancel unlocks; payment changes Amount Paid/Balance Due without changing Retainage released.

E2E-011 Customer PDF and web views reconcile to the same immutable invoice snapshot for tracked and release invoices.

E2E-012 QBO/network retry does not duplicate invoices, retainage transfers, payments, or external mappings.

11.3 Requirement traceability convention

- Product requirements: SYS, XF, PI, CI, CO, PS, RT, and RT-QBO identifiers in this document.
- Architecture requirements: ARC identifiers.
- Integrated acceptance scenarios: E2E identifiers.
- Each implementation ticket should cite requirement IDs, include its decision-record link, name affected aggregates/events, and list automated test IDs.
- AI-generated analysis should distinguish explicit requirements, approved architecture recommendations, open gaps, and deferred scope.

12. AI analysis guide

12.1 Answering precedence

Priority	Evidence
1	Explicit requirement IDs and system invariants in this document.
2	Feature-specific requirements and crossover tables in this document.
3	Approved answers to GAP items and architecture decision records created after this version.
4	Current prototype documentation and interactive behavior as examples, not production persistence/accounting proof.
5	Implementation details, only when consistent with higher-priority requirements.

12.2 Guardrails for AI-assisted work

- Do not infer production behavior from an inert prototype control or local fixture.
- Do not merge posted billing, Draft reservation, Payment Schedule projection, retainage withholding, retainage release, and payment into one balance.
- Do not treat Contract Invoice as a second invoice platform; treat it as contract-aware tracked billing.
- When an answer depends on a GAP, state the gap ID and request or propose a decision rather than silently choosing policy.
- When calculating money, state basis, included statuses, rounding point, and whether values are gross contract consumption or net A/R.
- Preserve internal/customer presentation differences and immutable saved-document snapshots.

12.3 High-value questions this document supports

- Can this user create a Standard, Progress, or contract-aware invoice in the current contract state, and why?
- Which totals change when a Draft is saved, an Invoice is sent, a CO is accepted, or Retainage is released?
- How should a new API transaction enforce one Draft and prevent stale allocation?
- What historical data must remain visible after a CO supersedes an already billed line?
- How does milestone reorder affect On acceptance automation and the invoice dropdown?
- What does a Retainage release change in contract billing, A/R, payments, customer documents, and QBO?
- Which open decisions block this implementation ticket and which deferred areas can remain outside scope?

12.4 Example structured query template

Prompt template

Using the Cinderblock Contract Billing Requirements v1.0: (1) cite the relevant requirement IDs; (2) identify authoritative persisted facts and derived values; (3) describe lifecycle and transaction consequences; (4) list affected crossover features; (5) flag unresolved GAP IDs; (6) propose acceptance tests; and (7) do not invent deferred behavior.

Appendix A. Source inventory and review record

A.1 Authoritative repository sources reviewed

Source	Role
AGENTS.md	Workspace instructions and dependency boundaries.
docs/current-prototype-state.md	Current module inventory, shared decisions, scenarios, validation, and working-tree cautions.
docs/progress-invoices-prototype.md	Tracked billing and Standard conversion requirements.
docs/contract-invoices-prototype.md	Accepted-CO contract billing requirements.
docs/change-orders-requirements.md	Change Order model, lifecycle, calculations, and acceptance checks.
docs/payment-schedule-prototype.md	Schedule hierarchy, editor, milestone billing, and acceptance automation.
docs/retainage-prototype.md	Retainage withholding, release, customer presentation, and QBO requirements.
docs/cinderblock-ui-system.md	Visual system and UI behavior boundaries.
docs/reference-review.md	Reference fidelity and document/customer surface distinctions.
Prototype model/README/test files	Implementation evidence, scenario coverage, calculations, and explicit non-production limitations.

A.2 Review pass 1 - semantic completeness

- Cross-checked every prototype section against the consolidated system invariants and feature crossover table.
- Separated Standard conversion, tracked gross billing, Draft reservation, schedule planning, retainage withholding/release, and payment semantics.
- Reconciled Contract Invoice under Progress Invoice conceptually while retaining source-aware requirements.
- Converted ambiguous or unimplemented production behavior into numbered gaps rather than unstated assumptions.
- Added readiness scoring with explicit evidence and limitations.

A.3 Review pass 2 - implementation and analysis fitness

- Checked that formulas identify their basis and included/excluded states.
- Checked that every cross-feature event has downstream consequences and transaction expectations.
- Checked that day-one data identities support history, reorder, retries, revisions, and integration reconciliation.
- Checked that roadmap phases have dependency-aware exit criteria and no UI phase precedes its required domain contract.
- Checked that deferred scope is explicit and does not hide unresolved P0/P1 business decisions.
- Checked document structure, requirement IDs, terminology, and AI query guidance for machine-assisted analysis.

A.4 Known evidence limits

- The document consolidates the current repository state; it does not claim approval of newly identified production decisions.
- Prototype behavior is frontend-only and local-state based. Visual interactions demonstrate intent, not persistence, security, accounting, or integration correctness.
- Readiness scores should be recalculated after P0/P1 decisions, service contracts, automated suites, and operational evidence exist.

End of requirements baseline